

Guía práctica y recomendaciones para FPGA-in-the-Loop

HDL Verifier para MATLAB/Simulink

Verificación de HDL con Simulink y FIL

Documento técnico

Autor: Juan Arzola

30 de diciembre de 2025

Índice general

1. Introducción	1
1.1. Estructura de un flujo FIL	1
1.2. Licencias y herramientas necesarias	1
2. Flujo operativo del asistente FIL	2
2.1. Iniciar el asistente	2
2.2. Configuración de la FPGA	3
2.3. Selección de fuentes HDL y configuración	5
2.4. Tipos de salida	6
2.5. Construcción del bloque FIL	6
2.6. Integración en Simulink	7
3. Bloque FIL en Simulink	8
3.1. Bloque FIL en Simulink: uso y conexión	8
3.1.1. Pestaña <i>Programming File</i>	9
3.1.2. Pestaña <i>Runtime Options</i>	9
3.1.2.1 Overclocking Factor	9
3.1.2.2 Output Frame Size	9
3.1.3. Atributos de señal	9
3.2. Ritmo de simulación y reloj del bloque FIL	10
3.3. Sample times y modos de simulación	10
3.3.1. Entradas discretas y errores comunes	10
3.3.2. Fixed-step vs. variable-step	11
3.4. Tipos de datos y conversión de buses	11
4. Ejemplos y casos de uso	13
4.1. Ejemplo de overclocking en un contador	13
4.1.1. Introducción	13
4.1.2. Caso de estudio	13
4.1.3. Configuración experimental	13
4.1.4. Resultados	14
4.1.5. Caso adicional: controlador PI	15

4.1.6.	Conclusión	18
4.2.	Ejemplos adicionales	18
4.3.	Ejemplo PID con anti-windup	18
5.	Restricciones y compatibilidad	19
5.1.	Restricciones	19
5.1.1.	Consideraciones de código HDL para objetos FIL	19
5.1.1.1	Requisitos generales	19
5.1.1.2	Tipos de puertos permitidos	19
5.1.1.3	Datos y construcciones no soportados	19
5.1.2.	Límites de datos	20
5.1.3.	Reglas de configuración del bloque FIL	20
5.1.4.	Selección de tipos de dato	20
5.1.5.	Resumen de compatibilidad	20

Índice de figuras

2.3	Selección de interfaz y frecuencia de reloj.	4
2.4	Validación de conexión con la FPGA.	5
2.6	Generación del bloque FIL desde el asistente.	6
3.1	Configuración de atributos de señal en el bloque FIL.	8
3.2	Configuración de atributos de señal en el bloque FIL.	9
3.3	Error al conectar una entrada continua a un bloque FIL discreto.	10
3.4	Error por discrepancia de tamaño de bus en la entrada del FIL.	11
3.5	Uso del bloque <i>Data Type Conversion</i> : conversión de entrada (izquierda) y parámetros de salida (derecha).	12
4.1	Diagrama de prueba en Simulink para el bloque FIL del contador.	14
4.2	Bloque FIL generado a partir del módulo counter	14
4.3	Comparación de salidas para distintos factores de overclocking.	15
4.4	Modelo PI base y su respuesta de referencia.	16
4.5	Diagrama de simulación con el bloque FIL del PI.	17
4.6	Respuesta del controlador PI para distintos factores de overclocking.	17

1. Introducción

FPGA-in-the-Loop (FIL) en HDL Verifier para MATLAB/Simulink es una técnica de verificación en la que el diseño HDL (DUT) se ejecuta en una FPGA física, mientras Simulink se utiliza como entorno de co-simulación para aplicar estímulos y recoger respuestas.

En este esquema, el objetivo es validar el comportamiento del HDL implementado ya en FPGA, en un entorno simulado más cercano a la realidad usando herramientas de MATLAB y Simulink. Esto permite comprobar funcionalidad, latencias y sincronización con señales de prueba antes de integrar el diseño en un sistema final.

1.1. Estructura de un flujo FIL

Un flujo FIL típico incluye:

- El DUT implementado en la FPGA y un bloque FIL en Simulink que actúa como interfaz de I/O.
- Un canal de comunicación entre Simulink y la FPGA (Ethernet, JTAG o PCIe, según la tarjeta).
- Un modelo de prueba en Simulink que genera estímulos, sincroniza el muestreo y observa las salidas.

1.2. Licencias y herramientas necesarias

Para ejecutar FIL se requieren licencias activas de:

- MATLAB y Simulink.
- HDL Verifier (incluye el `FIL Wizard` y el bloque FIL).
- HDL Coder, si el HDL se genera desde Simulink.
- Herramienta del fabricante para sintetizar y generar el bitstream (p. ej., Vivado para AMD/Xilinx o Quartus para Intel), junto con sus *support packages* cuando aplique.

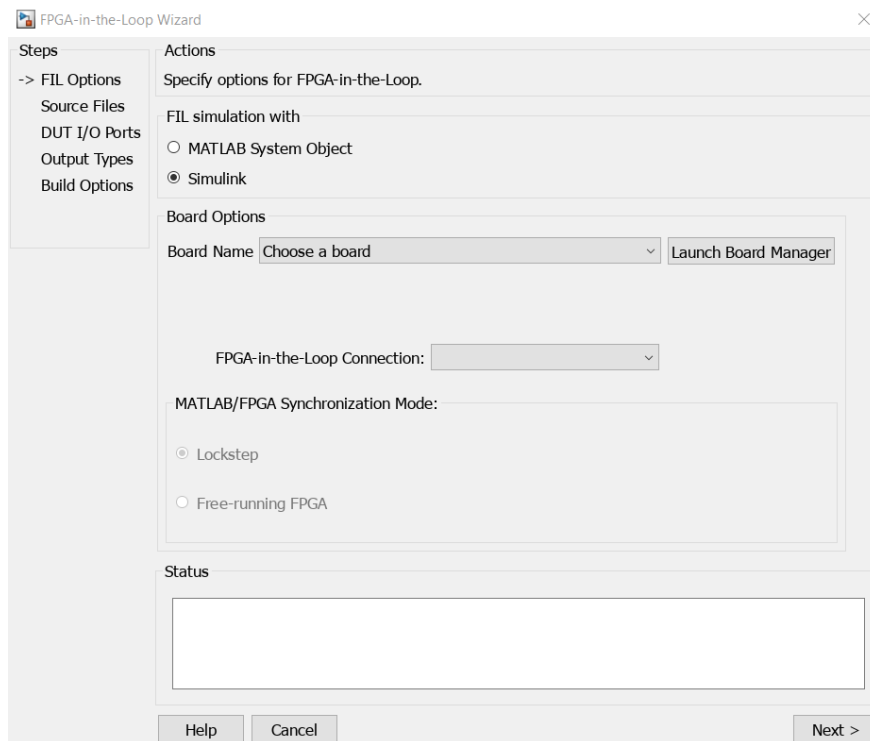
2. Flujo operativo del asistente FIL

2.1. Iniciar el asistente

1. En MATLAB, ejecute: `>> filWizard`.



(a) Llamada al asistente FIL desde la consola de MATLAB.



(b) Pantalla inicial del asistente FIL.

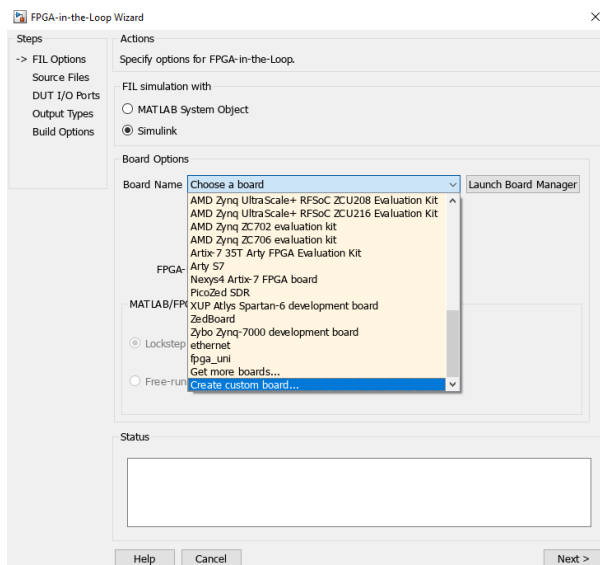
2. Elija **Simulink** como entorno de simulación.
3. Seleccione la tarjeta de desarrollo FPGA. Se recomienda instalar *HDL Verifier Support Package for AMD FPGA and SoC Devices* y *HDL Verifier Support Package for Intel FPGA Boards* para contar con más FPGAs predefinidas.
4. Defina el método de conexión soportado (Ethernet, JTAG, PCI Express o USB Ethernet) según la compatibilidad de la tarjeta y de HDL Verifier.

Dependiendo de la FPGA y el tipo de conexión, MATLAB se puede conectar con la FPGA en dos modos

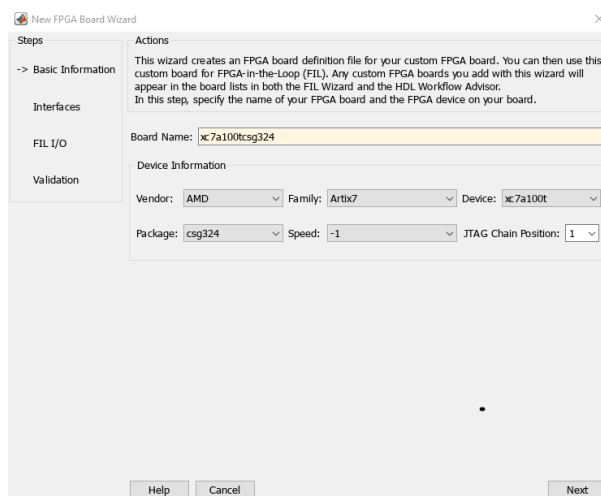
- **Lockstep (modo predeterminado):** La FPGA opera en sincronía con MATLAB/Simulink. Cada paso de simulación produce un ciclo de reloj en el hardware, logrando simulación ciclo a ciclo.
- **Free-Running:** La FPGA corre con su reloj interno y procesa datos de forma continua. MATLAB/Simulink solo intercambia datos bajo demanda y no gobierna los ciclos de reloj. Este modo solo está disponible en ciertas tarjetas y conexiones Ethernet.

2.2. Configuración de la FPGA

En esta etapa se definen los parámetros de la FPGA para que el FIL Wizard pueda generar y programar el diseño. En **Board name** se listan opciones predefinidas; si su FPGA no aparece, debe crearla desde **Create custom board** al final de la lista.



(a) Ventana inicial del FIL Wizard.



(b) Asistente de nueva placa FPGA.

Luego se define la interfaz con la que se conectará la FPGA. En este documento se utiliza **JTAG**, por lo que no se profundiza en las demás opciones; estas requieren que la FPGA tenga la interfaz física disponible o que se implemente en el diseño.

En **FPGA input clock** se debe ingresar la frecuencia del reloj esperado. En este caso se usa **100 MHz**; esta es la restricción con la que se sintetiza e implementa el diseño. Si el diseño no cumple temporización para esta frecuencia, la implementación fallará.

Steps

- Basic Information
- > Interfaces
- FIL I/O
- Validation

Actions

Specify the board interface. To enable FPGA-in-the-Loop, select the "FIL Interface" parameter.

Specify the clock and reset pin numbers. Parameter "Clock Frequency" must be between 5 and 300. For Ethernet interface, the suggested clock frequencies are 50, 100, 125, and 200MHz. Parameter "Clock Pin Number" must be specified. Leave the parameter "Reset Pin Number" empty if you do not have one. Here is an example of pin number assignment: N10.

FPGA-in-the-Loop Interface

☒ FIL Interface

PHY Interface type:

- ☒ JTAG
- ☐ Gigabit Ethernet - GMII
- ☐ Gigabit Ethernet - AMD SGMII
- ☐ Gigabit Ethernet - AMD SGMII with 625MHz Reference Clock
- ☐ Ethernet - RMII
- ☐ Ethernet - MII
- ☐ Ethernet - MII with 25MHz Output

Note: FPGA-in-the-Loop requires an HDL Verifier license. FPGA-in-the-Loop uses an Ethernet PHY chip or JTAG connector on the FPGA board to communicate with the host PC.

FPGA Input Clock

Clock Frequency: MHz Clock Type:

Clock Pin Number:

Clock I/O Standard:

Reset (Optional)

Reset Pin Number:

Active Level:

Reset I/O Standard:

Figura 2.3: Selección de interfaz y frecuencia de reloj.

En la opción de cadena JTAG, continúe con *Next* a menos que el JTAG no esté conectado directamente a la FPGA, es decir, si hay otros dispositivos en serie antes o después de la FPGA.

Por último, se abre una ventana para validar la conexión con la FPGA. Se recomienda activar las dos casillas y ejecutar la prueba. Si finaliza con éxito, la configuración es correcta para usar el FIL Wizard.

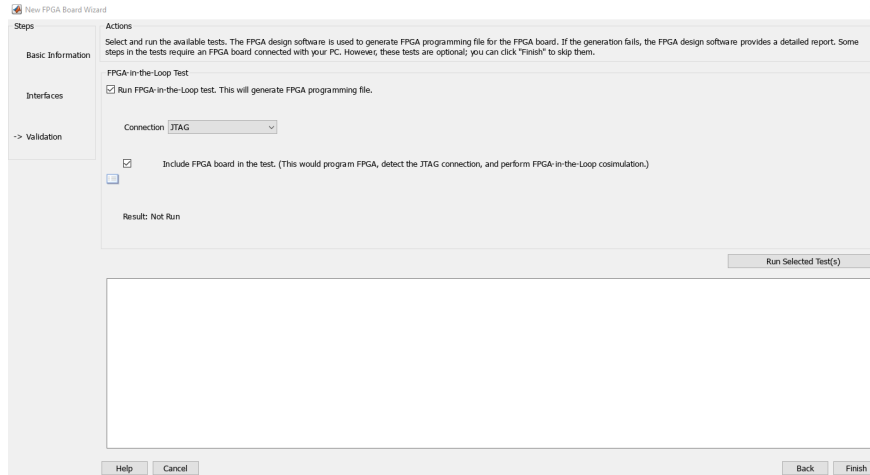
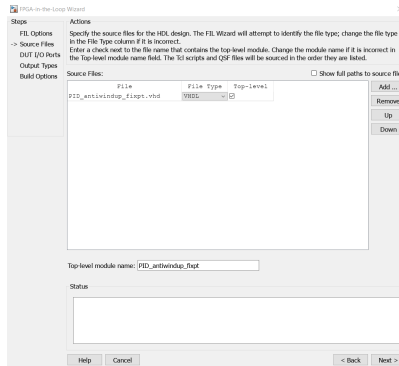
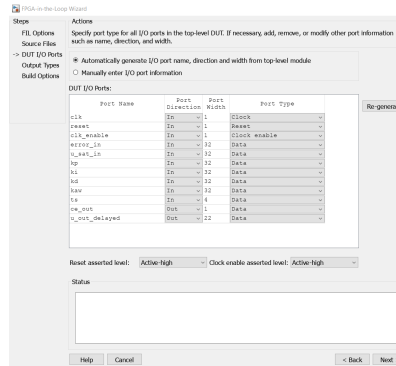


Figura 2.4: Validación de conexión con la FPGA.

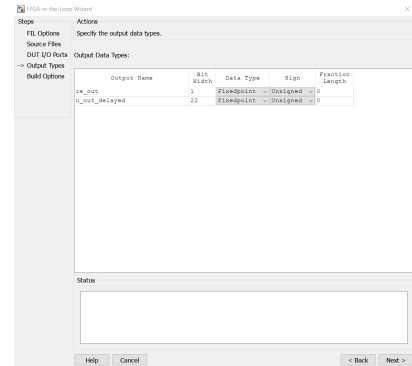
2.3. Selección de fuentes HDL y configuración



(a) Añadir archivos HDL.



(b) Tabla de puertos detectados.



(c) Configuración de salidas.

1. Añada los archivos HDL y declare el *top* del diseño.
2. Revise la tabla de puertos del DUT; el asistente analiza el módulo superior y muestra todas las E/S detectadas. Para entradas se contemplan cuatro casos: *clock*, *reset*, *clock enable* o *data*. Los tres primeros no generan puerto visible en el bloque FIL (se cablean internamente); las salidas solo pueden ser de tipo *data*. Los parámetros *reset asserted level* y *clock enable asserted level* dependen de si el diseño usa flanco positivo o negativo para reset y habilitación.
3. Ajuste manualmente cualquier discrepancia de tipo o tamaño en los puertos.

2.4. Tipos de salida

- Defina el tipo de dato que entregará el bloque FIL hacia Simulink: punto fijo, entero o lógico.
- Si detecta un tipo incorrecto, corríjalo manualmente antes de la generación; las salidas pueden modificarse después, pero las entradas no.

2.5. Construcción del bloque FIL

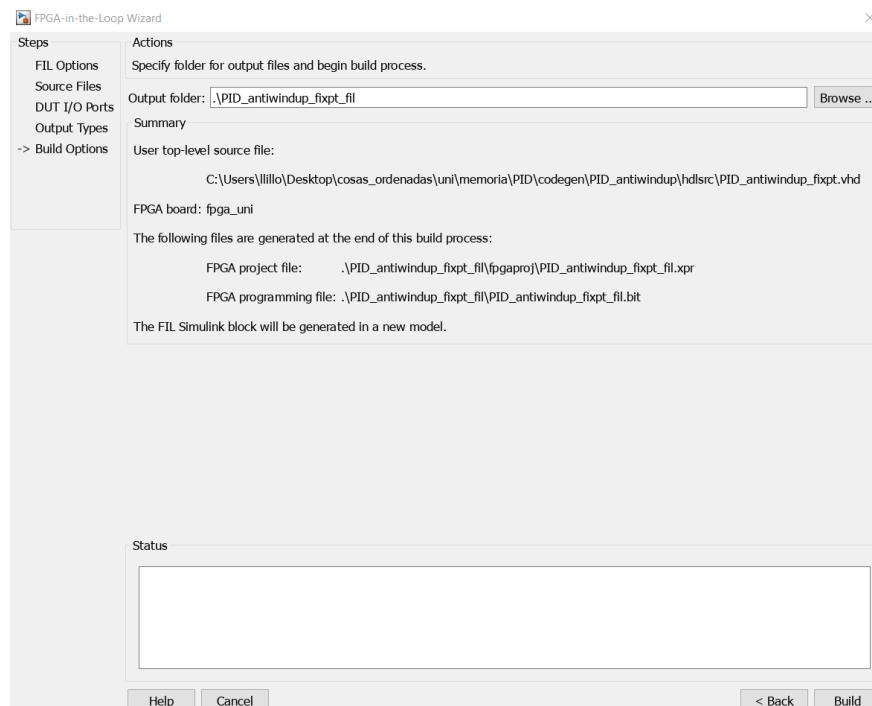


Figura 2.6: Generación del bloque FIL desde el asistente.

1. Indique la carpeta de salida para los artefactos generados.
2. Pulse **Build** para lanzar la generación del bloque FIL y el bitstream.
3. Una vez completada la síntesis, se abrirá un modelo de Simulink con el bloque FIL listo para integrar.
4. MATLAB debe tener en el *path* la ubicación del binario de Vivado/Altera. Ejemplo para agregarlo: `hdlsetuptoolpath('ToolName','Xilinx Vivado','ToolPath','C:\Xilinx\Vivado\202` (ajuste la versión según la instalada).

2.6. Integración en Simulink

- Al terminar la síntesis se crea un bloque FIL en Simulink y, en paralelo, inicia la implementación y generación del bitstream.
- Cargue el bitstream en la FPGA cuando el modelo se lo solicite.
- Conecte el bloque FIL al resto del diagrama y ejecute la simulación en el modo elegido.

3. Bloque FIL en Simulink

3.1. Bloque FIL en Simulink: uso y conexión

El bloque FIL generado por el asistente se inserta en el modelo de Simulink y se gestiona mediante dos pestañas principales. La Figura 3.1 muestra la interfaz general.

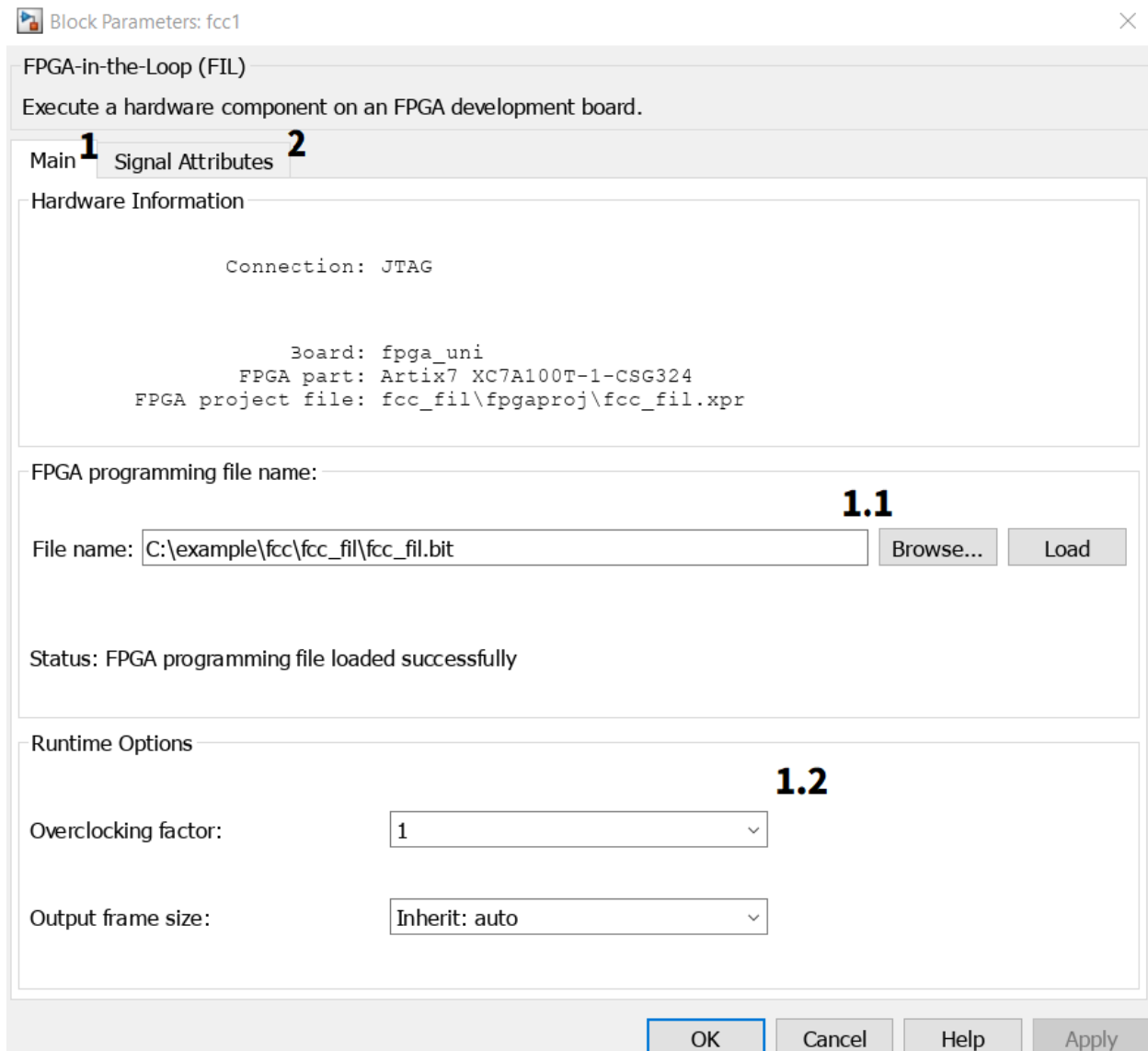


Figura 3.1: Configuración de atributos de señal en el bloque FIL.

3.1.1. Pestaña *Programming File*

- Cargue el bitstream generado al finalizar **filWizard** mediante la opción *Load*. La carga puede tardar varios minutos según el tamaño y la complejidad del bitstream.
- Espere a que concluya la programación antes de iniciar la simulación para evitar desalineaciones entre hardware y modelo.

3.1.2. Pestaña *Runtime Options*

3.1.2.1. Overclocking Factor

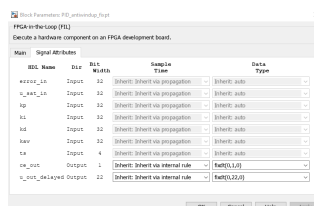
Configure cuántos ciclos de reloj de hardware se ejecutan por cada paso de simulación. Si el diseño HDL requiere n ciclos internos para producir una salida por cada entrada, establezca el *overclocking factor* en n . Un ejemplo práctico se describe en la Sección 4.1.

- Un factor insuficiente produce datos incompletos o inestables.
- Un factor excesivo introduce latencia adicional; en diseños con retroalimentación puede provocar acumulación de valores. Emplee señales **valid/ready** en el HDL para cortar el procesamiento cuando la operación ya finalizó.

3.1.2.2. Output Frame Size

Controla el tamaño de la trama de salida cuando se trabaja en modo *frame-based*. Permite alinear la cantidad de muestras emitidas por el bloque FIL con el ritmo esperado por el modelo aguas abajo. Internamente, se ajusta al sobre-reloj y al posible *downsample* de salida. Mantenga el valor por defecto cuando se trabajen señales escalar o *sample-based*; ajuste solo si el modelo requiere tramas de longitud específica.

3.1.3. Atributos de señal



entrada es continua, y salidas discretas si todas las entradas son discretas. También puede fijar un valor explícito (p. ej., $1e-3$).

- **Data type:** declare el tipo de dato de cada salida para que Simulink interprete correctamente la señal. Defina punto fijo o lógico cuando sea posible para trazabilidad y equivalencia con hardware.
- Las entradas se definen durante el asistente FIL; en el bloque solo se configuran salidas.

3.2. Ritmo de simulación y reloj del bloque FIL

- Por defecto, un paso de Simulink equivale a un ciclo de reloj del bloque FIL; en cada paso se leen salidas y se aplican nuevas entradas.
- Si el HDL necesita más de un ciclo para producir una salida, configure *overclocking factor* igual al número de ciclos internos requeridos. Esto evita submuestreo de la lógica.
- **Latencia (ciclos):** ciclos transcurridos desde la entrada hasta la salida. **Throughput:** resultados por ciclo. **Intervalo de iniciación (II):** número de ciclos entre entradas sucesivas; $\text{Throughput} = 1/\text{II}$.
- Use señales *valid/ready* en diseños con caminos de realimentación para frenar el procesamiento cuando la operación ha concluido, especialmente si el factor de sobre-reloj excede la latencia real.

3.3. Sample times y modos de simulación

3.3.1. Entradas discretas y errores comunes

El bloque FIL es discreto. Si recibe una entrada continua, Simulink mostrará un error como en la Figura 3.3. Defina un tiempo de muestreo en la ruta de entrada (p. ej., *Zero-Order Hold* o *Unit Delay*).

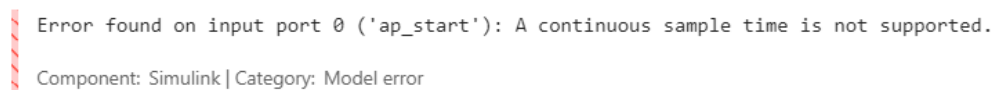


Figura 3.3: Error al conectar una entrada continua a un bloque FIL discreto.

Si una entrada tiene un sample time mínimo de T_{in} y el paso de simulación es menor, el bloque FIL toma datos a T_{in} , usando el menor tiempo de muestreo entre sus entradas como reloj efectivo.

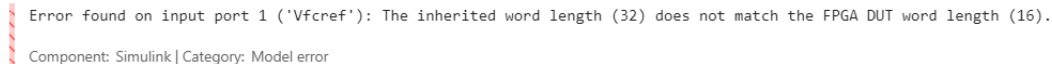
Si se necesita conectar el FIL con un sistema continuo en Simulink, se puede usar un *Zero-Order Hold* para simular un DAC junto con un bloque *Data Type Conversion* que convierta la salida a `double`. En sentido inverso, use otro *Data Type Conversion* para ajustar desde `double` al tipo de dato de entrada del FIL, junto con un *Zero-Order Hold* para simular un ADC.

3.3.2. Fixed-step vs. variable-step

- **Fixed-step:** recomendado para evitar desincronización. El overclocking debe ser múltiplo del paso fijo.
- **Variable-step:** necesario con dinámicas continuas. Todas las entradas al FIL deben estar discretizadas. Use *Zero-Order Hold* para entradas continuas, *Unit Delay* en la realimentación al propio FIL y, si la salida alimenta un bloque continuo, coloque un *Zero-Order Hold* en esa ruta. Mantenga el paso variable más pequeño que los sample times de entrada y de salida para preservar exactitud.

3.4. Tipos de datos y conversión de buses

El bloque FIL acepta buses de señales; no admite matrices como entrada. Si el ancho del bus no coincide con el de la señal, se produce un error como en la Figura 3.4.



Error found on input port 1 ('Vfcresf'): The inherited word length (32) does not match the FPGA DUT word length (16).
Component: Simulink | Category: Model error

Figura 3.4: Error por discrepancia de tamaño de bus en la entrada del FIL.

Para adaptar tipos, use el bloque *Data Type Conversion* (Figuras 3.5). Configure el tipo de salida como `fixdt` o lógico según corresponda. Ejemplos:

- `fixdt(1, 32, 0)` equivale a `int32`.
- `fixdt(0, 16, 0)` equivale a `uint16`.
- `fixdt(1, 16, 8)` es un punto fijo con signo de 16 bits y 8 bits fraccionarios.

Para los límites generales de tamaño y coherencia de señal, consulte la Sección 5.1.

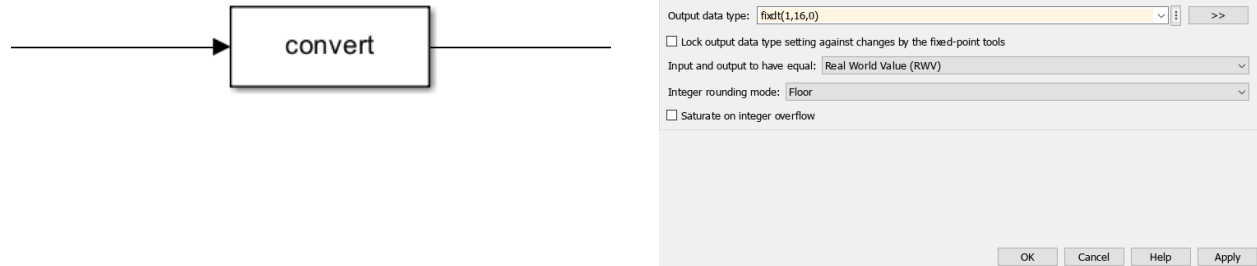


Figura 3.5: Uso del bloque *Data Type Conversion*: conversión de entrada (izquierda) y parámetros de salida (derecha).

Si una salida del bloque FIL se retroalimenta a una entrada, inserte un *Zero-Order Hold* o un *Unit Delay* en ese lazo. Simulink no permite un ciclo directo con un bloque discreto sin discretizar la ruta de realimentación, por lo que es obligatorio introducir ese elemento para cerrar el lazo.

4. Ejemplos y casos de uso

4.1. Ejemplo de overclocking en un contador

4.1.1. Introducción

Este ejemplo documenta la verificación de un módulo Verilog sencillo mediante FPGA-in-the-Loop (FIL). El caso de estudio es un contador con varias salidas que incrementan a ritmos distintos y una entrada `start` que habilita el conteo. Se analiza cómo el *overclocking factor* (OF) afecta el muestreo de entradas y la lectura de salidas en Simulink.

El *overclocking factor* se define como la razón entre la frecuencia de reloj del FPGA y la frecuencia de muestreo de Simulink. Para un periodo de muestreo T_s :

$$f_{FPGA} = \frac{OF}{T_s} \quad \text{y} \quad T_{FPGA} = \frac{T_s}{OF}$$

Esto implica que el FPGA muestrea las entradas `OF` veces por cada paso de Simulink, y la salida observada ya incluye `OF` ciclos internos acumulados.

4.1.2. Caso de estudio

El módulo `counter` recibe `clk`, `rst`, `start` e `inc`. Internamente implementa un contador base y divisores de ciclo que generan pulsos de incremento. Las salidas son contadores de 8 bits con diferentes frecuencias:

- `cnt1`: incrementa en cada flanco cuando `start` está activo.
- `cnt10`: incrementa cada 10 ciclos.
- `cnt17`: incrementa cada 17 ciclos.

El objetivo es observar cómo el `OF` interactúa con estas frecuencias internas y cómo se refleja en la simulación.

4.1.3. Configuración experimental

Se usa `FIL Wizard` para generar el bloque FIL desde `counter.v` y se integra en Simulink con fuentes y *scopes*. La Figura 4.1 muestra el diagrama del experimento.

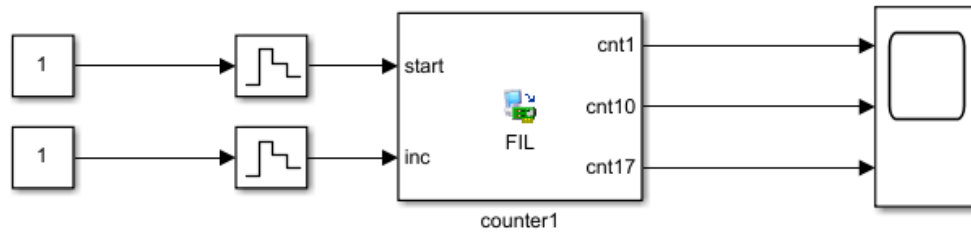


Figura 4.1: Diagrama de prueba en Simulink para el bloque FIL del contador.

El muestreo de entradas se fija en $T_s = 1 \times 10^{-2}$ s y se evalúan los casos $OF = 1$, $OF = 10$ y $OF = 17$ en un horizonte de 1 s.

4.1.4. Resultados

La Figura 4.2 muestra el bloque FIL generado. La Figura 4.3 compara los resultados de los *scopes* para distintos factores de overclocking.

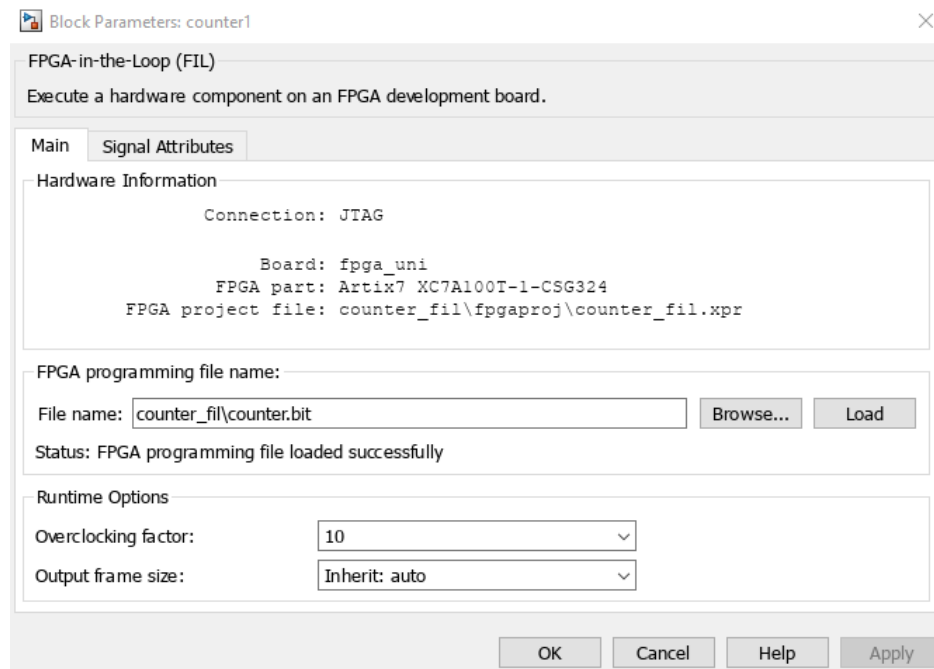


Figura 4.2: Bloque FIL generado a partir del módulo counter.

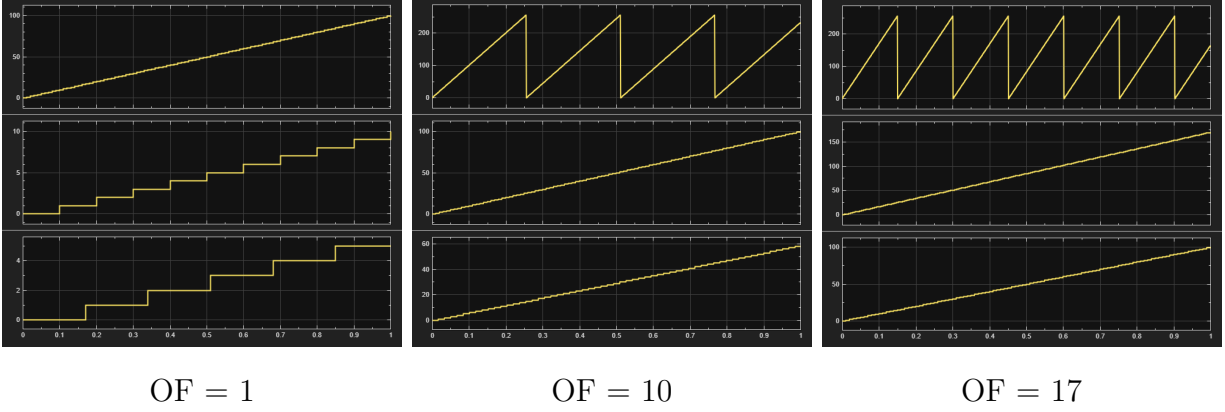


Figura 4.3: Comparación de salidas para distintos factores de overclocking.

En un intervalo de 1 s, el comportamiento observado se resume así:

- OF = 1: el contador de 1 ciclo llega a 100; los de 10 y 17 ciclos quedan por debajo por falta de ciclos internos.
- OF = 10: el contador de 10 ciclos alcanza 100; el de 1 ciclo supera 100 y el de 17 ciclos queda por debajo.
- OF = 17: el contador de 17 ciclos alcanza 100; los de 1 y 10 ciclos superan 100 por acumulación interna.

4.1.5. Caso adicional: controlador PI

Para un escenario más cercano a una aplicación real, se evalúa un controlador PI donde la parte integral acumula el error. En estos sistemas, el sobremuestreo o el muestreo insuficiente alteran el comportamiento esperado.

Se parte de un modelo base en Simulink con una función MATLAB que implementa el PI. La Figura 4.4 muestra el diagrama y la respuesta de referencia.

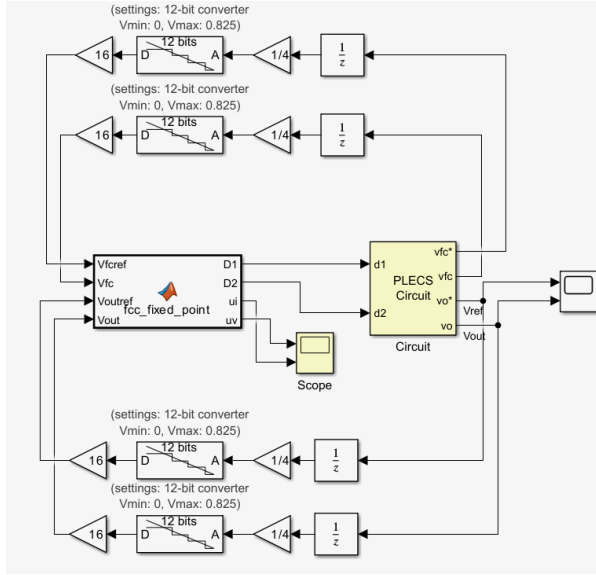
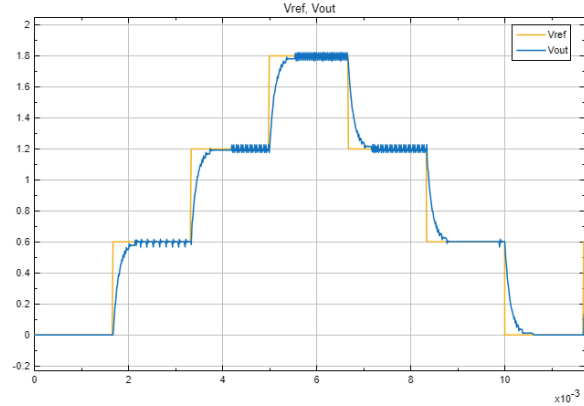


Diagrama base del PI



Respuesta del PI base

Figura 4.4: Modelo PI base y su respuesta de referencia.

Las entradas se discretizan a 6.5×10^{-6} s. Dado que PLECS no permite modo *discrete*, se usa *variable step* y un bloque **Zero-Order Hold** para conectar el bloque FIL (discreto) con el dominio continuo de PLECS. La salida de la FPGA se convierte a **int8** para empatar el ancho de palabra.

El controlador se sintetiza en HDL con Vitis y luego se genera el bloque FIL con **FIL Wizard**. El HDL incluye señales de control (p. ej., **D1_ap_vld**) que permiten inferir el número de ciclos internos necesarios. En este caso, la señal indica una salida cada 7.15×10^{-5} s, equivalente a 11 ciclos internos por salida, por lo que se espera un **OF = 11**. La Figura 4.5 muestra el diagrama con FIL.

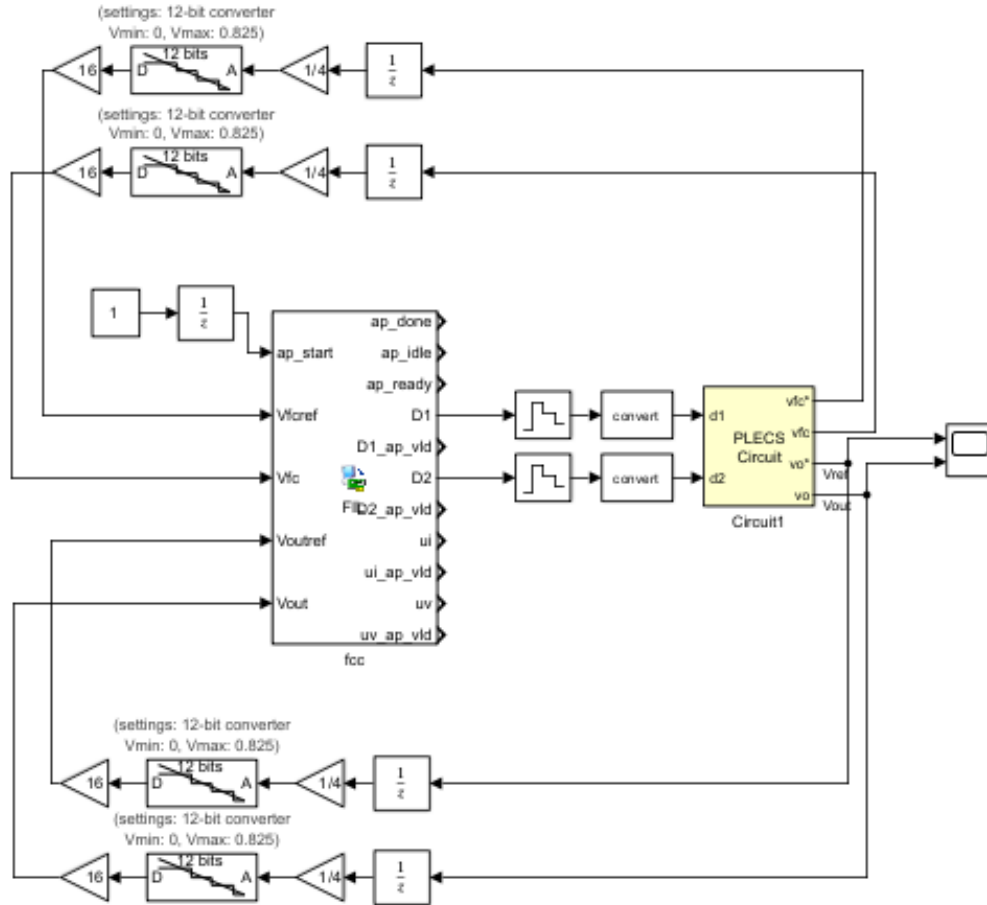


Figura 4.5: Diagrama de simulación con el bloque FIL del PI.

Se evalúan tres factores de overclocking: $OF = 1$, $OF = 11$ y $OF = 75$. Los resultados se muestran en la Figura 4.6.

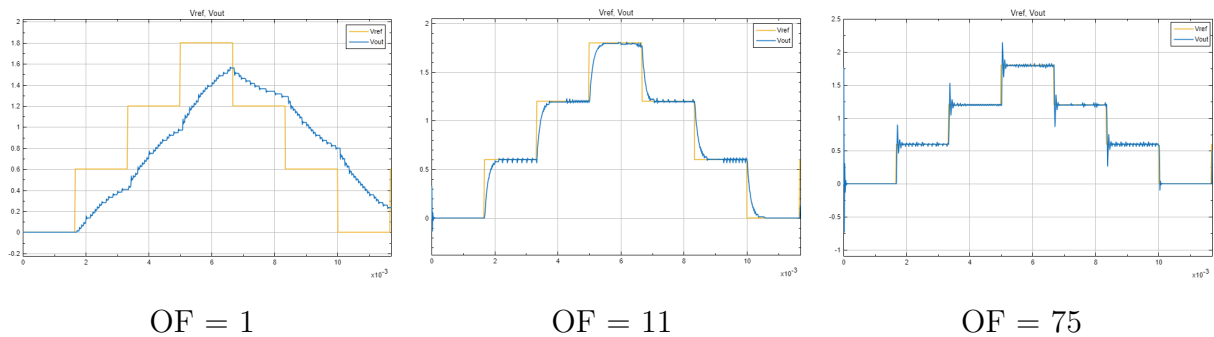


Figura 4.6: Respuesta del controlador PI para distintos factores de overclocking.

Los resultados por factor de overclocking son:

- $OF = 1$: el controlador no logra el desempeño esperado porque requiere 11 ciclos internos

para producir una salida válida.

- **OF = 11:** la respuesta coincide con la referencia al respetar los ciclos internos requeridos.
- **OF = 75:** se evidencia sobremuestreo, con sobreimpulsos más pronunciados y mayor tiempo de estabilización.

4.1.6. Conclusión

El **OF** debe seleccionarse según la frecuencia interna del módulo para obtener resultados coherentes en FIL. En este contador, el valor de **OF** que mejor reproduce un incremento de 100 en 1 s coincide con la frecuencia de conteo del canal observado. Un **OF** alto encapsula más actualizaciones internas en cada paso de Simulink y puede inflar los valores si el diseño no limita la acumulación. Además, cuando la frecuencia de actualización del FPGA difiere del muestreo en Simulink, aparecen saltos visibles y valores finales inferiores a los esperados.

4.2. Ejemplos adicionales

4.3. Ejemplo PID con anti-windup

Los archivos del ejemplo PID con anti-windup no están disponibles en esta carpeta.

5. Restricciones y compatibilidad

5.1. Restricciones

5.1.1. Consideraciones de código HDL para objetos FIL

5.1.1.1. Requisitos generales

- La entidad o módulo superior debe estar en VHDL, Verilog o SystemVerilog.
- El nombre del archivo HDL debe coincidir con el nombre de la entidad o módulo superior.
- Se requiere al menos un puerto de salida y todos los puertos de E/S deben declararse en la interfaz.
- Los diseños secuenciales deben tener un único reloj en el nivel superior. Declare reloj y reset. Para diseños combinacionales no es obligatorio el bundle de reloj.
- Nombre el reloj como `clock` o `clk`. Si usa otro nombre, márkelo en el asistente de FIL. El puerto de reloj debe ser de un bit y, en VHDL, de tipo `std_logic`.

5.1.1.2. Tipos de puertos permitidos

- **VHDL:** `std_logic`, `std_logic_vector`.
- **Verilog/SystemVerilog:** `reg`, `wire`, `logic`, `bit`, `byte`, `shortint`, `int`, `longint`, `real`, `shortreal`, arreglos empaquetados de `reg`, `wire` y `logic`.
- Los rangos de puertos vectoriales deben ser descendentes (ej. `9 DOWNT0 0` o `9:0`) y literales. No se admiten genéricos (VHDL) ni parámetros (Verilog/SystemVerilog) para definir el rango.
- Para Verilog, los nombres de puertos y del módulo deben estar en minúsculas.

5.1.1.3. Datos y construcciones no soportados

- Puertos bidireccionales, arreglos o registros (*record*) en VHDL/Verilog.
- Tipos enumerados, estructuras, uniones y arreglos no empaquetados en SystemVerilog.
- Sentencias de configuración en VHDL, ficheros de inclusión en Verilog, macros, nombres escapados y puertos duplicados.

- `typedef` en SystemVerilog.

5.1.2. Límites de datos

- El tamaño total de datos de entrada debe ser menor a 1467 bytes (suma de los bits de entrada redondeada al siguiente byte).
- El tamaño total de datos de salida también debe ser menor a 1467 bytes (suma de los bits de salida redondeada al siguiente byte).
- Tamaño de trama de salida: $\text{OutputFrameSize} = \text{InputFrameSize} \times \text{OverclockingFactor} / \text{OutputDownsample}$.

5.1.3. Reglas de configuración del bloque FIL

- Las señales deben tener el mismo ancho de bit que sus puertos correspondientes en hardware.
- Al procesar por tramas, todas las entradas deben compartir el mismo tiempo de muestreo; todas las salidas, también. El tiempo de muestreo de salida puede diferir del de entrada.
- Asegure que exista al menos un puerto de salida en el diseño superior.

5.1.4. Selección de tipos de dato

- El bloque FIL acepta enteros con signo, sin signo, enteros largos y booleanos.
- Se recomienda usar **punto fijo** (`fixdt`) y **lógico** para mayor robustez y trazabilidad. Equivalencias comunes:
 - `fixdt(1, 32, 0)` equivale a `int32`.
 - `fixdt(0, 16, 0)` equivale a `uint16`.
 - `fixdt(1, 16, 8)` define un punto fijo con signo de 16 bits y 8 bits fraccionarios.
 - `fixdt(0, 8, 0)` equivale a `uint8` para buses de control o banderas simples.
- Use **lógico** para buses de control o flags (p. ej., un bus de 64 bits que transporta señales booleanas).

5.1.5. Resumen de compatibilidad

- No se admiten puertos bidireccionales ni estructuras complejas (records, structs, unions, enumeraciones, arreglos no empaquetados).

- Emplee rangos descendentes literales y evite genéricos o parámetros para dimensionar buses.
- Mantenga los nombres de puertos y módulos en minúsculas al trabajar con Verilog.